Reporte Técnico de Actividades Práctico-Experimentales Nro 4.

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Ivett Zaragocin, Anthony Yaguana, Pedro Jiménez, Steven Pardo
Asignatura	Simulación
Ciclo	5
Unidad	1
Resultado de aprendizaje de la unidad	**R1.** Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	4
Título de la Práctica	Análisis comparativo de herramientas de simulación mediante un modelo común
Nombre del Docente	José O. Guamán Q.
Fecha	22 de Octubre de 2026 23 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

Comparar diferentes lenguajes y plataformas para la implementación de modelos de simulación, a partir de un modelo epidemiológico SIR común.

- Implementar un modelo SIR sencillo en Python con SimPy y en NetLogo.
- Comparar la sintaxis, la facilidad de implementación y la visualización de cada herramienta.
- Identificar el ámbito de aplicación de cada herramienta.

3. Materiales, Reactivos, Equipos y Herramientas

- Computador.
- Python 3.12.
- Biblioteca SimPy 4.1.
- Matplotlib 3.11.
- NetLogo 7.0 con la extensión csv.
- Git y GitHub.
- Editor de código: Visual Studio Code.
- Sistema operativo Linux.

4. Procedimiento / Metodología Ejecutada

Fundamento teórico

El modelo SIR, propuesto por Kermack y McKendrick [1], es un modelo compartimental que describe la propagación de una enfermedad infecciosa en una población cerrada. Cada individuo pertenece a uno de tres compartimentos: susceptibles (S), que todavía pueden contagiarse; infectados (I), que tienen la enfermedad y pueden transmitirla; y recuperados (R), que ya superaron la infección y no vuelven a contagiarse [2]. Como la población no cambia de tamaño durante la simulación, en todo instante se cumple:

$$N = S + I + R \quad (1)$$

La evolución del sistema depende de dos parámetros: la tasa de transmisión β, que mide la probabilidad de contagio en un contacto entre un infectado y un susceptible, y la tasa de recuperación γ, que mide la fracción de infectados que se recupera por unidad de tiempo [2]. En su forma continua, el modelo se expresa con el siguiente sistema de ecuaciones diferenciales:

$$dS/dt = -\beta \cdot S \cdot I / N \quad (2)\ \textit{variación de susceptibles}$$

$$dI/dt = \beta \cdot S \cdot I / N - \gamma \cdot I \quad (3)\ \textit{variación de infectados}$$

$$dR/dt = \gamma \cdot I \quad (4)\ \textit{variación de recuperados}$$

El término $\beta \cdot S \cdot I/N$ representa los nuevos contagios por unidad de tiempo y $\gamma \cdot I$ los infectados que se recuperan. Así, el flujo de individuos es siempre $S \rightarrow I \rightarrow R$: un susceptible se infecta ($S + I \rightarrow I + I$) y un infectado se recupera ($I \rightarrow R$).

El número reproductivo básico indica cuántas infecciones produce, en promedio, un infectado al inicio de la epidemia, cuando casi toda la población es susceptible [2]:

$$R_0 = \beta / \gamma = 0.30 / 0.10 = 3 \quad (5)$$

Como $R_0 > 1$, se espera que la infección se propague: el número de infectados crece, alcanza un máximo y luego disminuye a medida que se agotan los susceptibles [2].

En lugar de resolver las ecuaciones de forma analítica, la práctica usa una simulación discreta basada en individuos: el tiempo avanza en pasos de una unidad y en cada paso se decide al azar quién se contagia y quién se recupera [3]. Por eso, los resultados dependen del generador de números aleatorios. Para que sean reproducibles se fija una semilla, que hace que el generador produzca siempre la misma secuencia [3].

Se utilizaron dos herramientas. SimPy es una biblioteca de Python para simulación de eventos discretos basada en procesos: cada proceso es una función generadora que avanza el reloj de la simulación con env.timeout() [4]. NetLogo es un entorno de modelado basado en agentes, en el que cada individuo es un agente (turtle) con su propio estado y la simulación avanza por ticks [5]. Python genera los datos y los exporta en un archivo CSV, que NetLogo importa para visualizarlos y compararlos con su propio modelo.

La tabla 1 resume los parámetros utilizados, que son los mismos en ambas herramientas.

Parámetro	**Símbolo**	**Valor**
Población total	N	100
Infectados iniciales	I_0	5
Susceptibles iniciales	S_0	95
Tasa de infección	β	0.30
Tasa de recuperación	γ	0.10
Tiempo de simulación	t	0 a 100
Semilla aleatoria	—	42
Número reproductivo básico	R_0	3

Tabla 1. Parámetros del modelo SIR

Metodología ejecutada

La práctica se desarrolló en Python 3.12 con SimPy y Matplotlib, y en NetLogo 7.0, siguiendo estas etapas:

- **Análisis de la guía.** Se revisaron el modelo SIR, sus parámetros, el pseudocódigo común y el formato del archivo CSV que NetLogo debe importar. Por acuerdo del grupo, la comparación se realizó entre Python/SimPy y NetLogo, sin la implementación en Java.
- **Acuerdo de reglas comunes.** Antes de programar se fijaron los datos que todos usan (tabla 1), la regla de cada paso, en la que cada infectado elige un contacto al azar, y la cabecera del CSV con las columnas tiempo, susceptibles, infectados y recuperados.
- **Creación del repositorio.** Se creó el repositorio en GitHub (github.com/ivecanahi/ape4-grupal) con la carpeta python/ y el archivo requirements.txt.
- **División del trabajo.** La parte en Python se dividió en cuatro partes (tabla 2): datos y población, simulación con SimPy, salida de resultados, e integración y pruebas. La implementación en NetLogo se realizó de forma individual.
- **Desarrollo con pruebas.** Se escribieron 13 pruebas automatizadas con unittest que verifican el modelo, el archivo CSV, la gráfica y el reporte.

- **Integración y corrección.** Al integrar las partes se detectó que el controlador importaba un módulo que había cambiado de nombre, por lo que el programa no se ejecutaba. Se unieron la Parte 1 y la Parte 2 en el archivo model.py, como indica la guía, y se unificó la forma de importar los módulos.
- **Reporte visual.** Se añadió una vista que genera un reporte HTML con la explicación de la práctica, ilustraciones del modelo, las métricas, la gráfica y la tabla de datos.
- **Integración con NetLogo.** Se copió resultados_python.csv a la carpeta del modelo de NetLogo y se importó con el botón CARGAR PYTHON para compararlo con el modelo propio.
- **Ejecución y análisis.** Se ejecutó la simulación, se analizaron las curvas S(t), I(t) y R(t) y se compararon las dos herramientas.

Parte	Archivo	Contenido
1. Datos y población	python/model.py (parte superior)	State, SIRParameters, StepResult y Population.
2. Simulación con SimPy	python/model.py (parte inferior)	SIRSimulation: registro, contagio, recuperación y proceso de SimPy.
3. Salida	python/view.py	CsvExporter, ChartView y ReportView.
4. Integración y pruebas	controller.py, main.py y test_model.py	SIRController, programa principal y pruebas.

Tabla 2. División del trabajo en Python

El proyecto sigue el patrón Modelo-Vista-Controlador (MVC): el modelo calcula la epidemia, las vistas presentan los resultados y el controlador conecta ambos. Así, la generación de datos queda separada de su visualización. La tabla 3 describe los módulos.

Módulo	Función
python/model.py	Modelo: estados, parámetros, población y simulación SIR con SimPy.
python/view.py	Vistas: exporta el CSV y genera la gráfica S(t), I(t), R(t) y el reporte HTML.
python/controller.py	Controlador: ejecuta la simulación y entrega los resultados a cada vista.
python/main.py	Programa principal: ejecuta todo y abre el reporte en el navegador.
python/test_model.py	Pruebas del modelo y del controlador.
python/test_view.py	Pruebas del CSV, la gráfica y el reporte.

Tabla 3. Módulos del proyecto en Python

La simulación en Python sigue el pseudocódigo de la guía y funciona así:

- Se crea un generador de números aleatorios random.Random(42) y una población de 100 individuos en estado S; con rng.sample se eligen 5 al azar y se pasan a I.
- Se crea el entorno de SimPy (simpy.Environment) y un proceso que se repite desde t = 0 hasta t = 100.
- En cada paso se registran el tiempo y los conteos de S, I y R, y se toma una copia del estado de la población.
- Cada individuo que estaba infectado en la copia elige un contacto al azar, distinto de sí mismo. Si el contacto era susceptible y un número aleatorio es menor que $\beta = 0.30$, se contagia.
- Después, cada individuo que estaba infectado en la copia se recupera si un número aleatorio es menor que $\gamma = 0.10$.
- El proceso ejecuta yield env.timeout(1) para avanzar el reloj una unidad. Al llegar a t = 100 se registra la última fila, con lo que se obtienen 101 registros.
- Al terminar, el controlador guarda resultados_python.csv, la gráfica grafica_sir_python.png y el reporte reporte_python.html.

Los cambios se calculan sobre la copia del estado para que un individuo contagiado en un paso no pueda contagiar a otro en ese mismo paso.

En NetLogo se creó el modelo: modelo_sir_netlogo.nlogox, con 100 agentes (turtles) y los mismos parámetros de la tabla 1, que también se pueden cambiar con deslizadores. El procedimiento setup fija random-seed 42, crea los agentes en estado S y elige 5 al azar como infectados. En cada tick, go guarda una copia del estado de cada agente (estado-previo); cada agente que estaba infectado elige un contacto al azar con one-of other turtles, lo contagia si era susceptible y random-float 1 es menor que β, y después se recupera si random-float 1 es menor que γ.

A diferencia de Python, que procesa primero todos los contagios y luego todas las recuperaciones, NetLogo aplica ambos pasos a cada infectado, en el orden literal del pseudocódigo de la guía. Como en los dos casos las decisiones se toman sobre la copia del estado, el modelo es el mismo; solo cambia el orden en que se usan los números aleatorios.

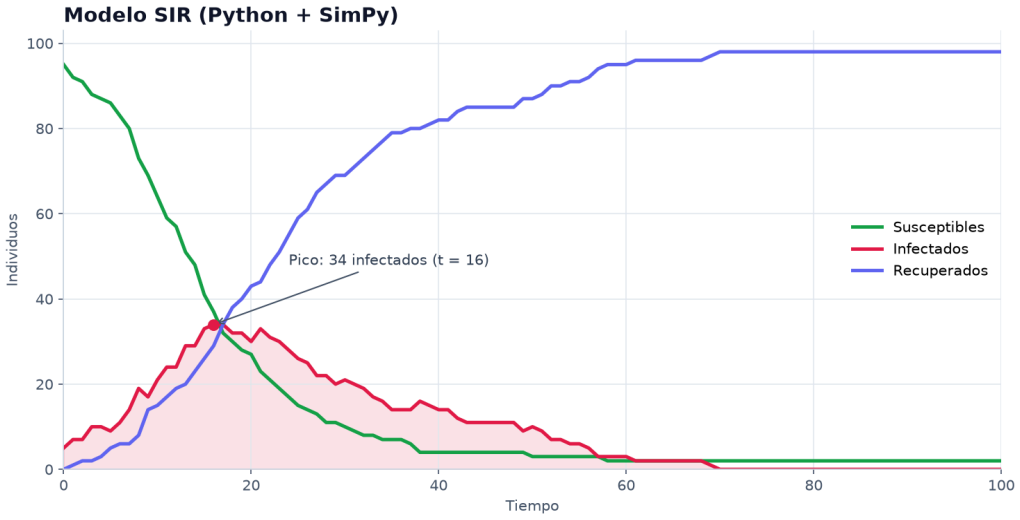
El botón CARGAR PYTHON abre resultados_python.csv, El modelo incluye también el botón CARGAR JAVA, que no se usó porque el grupo no realizó la implementación en Java.

Para reproducir los resultados se ejecuta pip install -r requirements.txt y luego python main.py dentro de la carpeta python/. El programa imprime el estado final y genera los tres archivos de salida. Las pruebas se ejecutan con python -m unittest.

5. Resultados

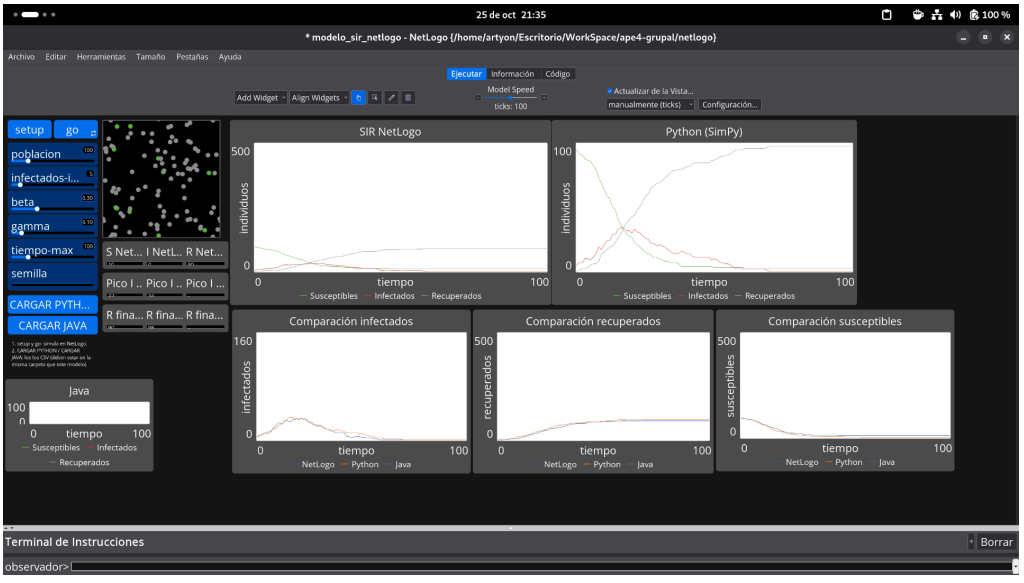
- **Resultados en Python**

  La figura 1 muestra la evolución obtenida con Python y la semilla 42. Al inicio la epidemia crece con rapidez: en t = 10 ya hay 21 infectados y los susceptibles bajaron de 95 a 64. El pico se alcanza en t = 16 con 34 infectados, el 34 % de la población. Desde ese punto los infectados van disminuyendo, porque quedan pocos susceptibles a quienes contagiar, y la infección se extingue en t = 70. Al final, 98 individuos se recuperaron y solo 2 nunca se contagiaron.



- **Resultados en NetLogo**

  Con la misma semilla, el modelo de NetLogo alcanzó un pico de 32 infectados en t = 19. La infección terminó en t = 59, con 90 recuperados y 10 susceptibles que no llegaron a contagiarse. Al pulsar CARGAR PYTHON, NetLogo importó las 101 filas del CSV y las dibujó junto a sus propias curvas.



- **Comparación de herramientas**

Característica	**Python/Simpy**	**NetLogo**
Facilidad de Implantación	Media, porque hubo que programar casi todo incluido el registro de resultados; SimPy solo aporta el reloj y el proceso.	Alta, porque los agentes, sus estados y el avance por ticks ya están incluidos (create-turtles, ask, tick); el modelo cabe en pocas líneas.
Manejo de agentes	De manera manual, los individuos son una lista de estados dentro de la clase Population.	Es nativo, cada individuo es una turtle con variables propias y se le dan órdenes con ask.
Generación de datos	Directa, CsvExporter genera el CSV con la cabecera común y 101 filas.	Es posible con listas y export-plot, pero en la práctica solo importó el CSV de Python.
Visualización	Con la librería Matplotlib, gráfica PNG y reporte HTML generados al terminar la simulación.	Integrada y en directo: vista de agentes, gráficas y monitores que se actualizan en cada tick.
Flexibilidad	Alta, se tiene todo el ecosistema de Python.	Limitada al lenguaje de NetLogo y a sus extensiones.
Tiempo de ejecución	≈ 1.3 ms por corrida.	≈ 11.9 ms por corrida.
Integración con otras herramientas	Exporta un CSV estándar que cualquier otra herramienta puede leer.	Importa archivos CSV con la extensión csv y puede ejecutar el código Python con la extensión py.

6. Preguntas de Control

1. ¿Qué representa el modelo SIR?

Es un modelo matemático utilizado para representar la evolución de una población durante la propagación de una infección. L

2. ¿Qué representan los estados S, I y R?

S: Individuos susceptibles, **I:** Individuos infectados, **R:** Individuos recuperados.

3. ¿Qué función cumplen los parámetros β y γ?

β (beta): tasa de transmisión o infección, **γ (gamma):** tasa de recuperación.

4. ¿Por qué es importante utilizar los mismos parámetros en las tres herramientas?

Para que la comparación sea justa. Si los parámetros cambian, no se sabría si una diferencia se debe a la herramienta o a los datos de entrada.

5. ¿Por qué Python, Java y NetLogo pueden producir resultados diferentes utilizando el mismo modelo?

Porque la simulación es estocástica y cada herramienta tiene su propio generador de números aleatorios.

6. ¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?

Cada individuo es un agente con su propio estado, y permiten escribir las reglas en pocas líneas. Además, la vista, las gráficas y los monitores muestran la simulación en tiempo real sin necesidad de programación adicional.

7. ¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?

SimPy controla el reloj y los procesos de la simulación: cada proceso avanza el tiempo.

8. ¿Qué ventajas ofrece Java para implementar modelos de simulación?

No se implemento en Java.

9. ¿Qué función cumple el archivo CSV en la integración Python/Java → NetLogo?

Es el medio de intercambio de datos, Python guarda los resultados en resultados_python.csv y NetLogo los lee para graficarlos y compararlos.

10. ¿Qué ventajas presenta separar la generación de datos de la visualización?

Cada parte se puede modificar y probar por separado, y los mismos datos sirven para varias salidas: el CSV, la gráfica, el reporte HTML y NetLogo.

7. Conclusiones

- El modelo SIR se implementó con los mismos parámetros en Python con SimPy y en NetLogo. Ambos reproducen la dinámica esperada: los infectados crecen, alcanzan un pico entre t = 16 y t = 19 y luego desaparecen.
- NetLogo facilitó representar y visualizar a los agentes en tiempo real, mientras que Python con SimPy ofreció más control, pruebas automatizadas y una ejecución unas nueve veces más rápida. El archivo CSV permitió integrar ambas herramientas sin que una dependa de la otra.

Referencias

[1]W. O. Kermack y A. G. McKendrick, "A contribution to the mathematical theory of epidemics," *Proceedings of the Royal Society of London. Series A*, vol. 115, no. 772, pp. 700–721, 1927, doi: 10.1098/rspa.1927.0118.

[2]H. W. Hethcote, "The mathematics of infectious diseases," *SIAM Review*, vol. 42, no. 4, pp. 599–653, 2000, doi: 10.1137/S0036144500371907.

[3]J. Banks, J. S. Carson, B. L. Nelson y D. M. Nicol, *Discrete-Event System Simulation*, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[4]Team SimPy, "SimPy: Discrete event simulation for Python," *SimPy Documentation*. [En línea]. Disponible en: https://simpy.readthedocs.io/

[5]U. Wilensky, "NetLogo," Center for Connected Learning and Computer-Based Modeling, Northwestern University, Evanston, IL, USA, 1999. [En línea]. Disponible en: https://ccl.northwestern.edu/netlogo/

[6]M. Matsumoto y T. Nishimura, "Mersenne Twister: A 623-dimensionally equidistributed uniform pseudo-random number generator," *ACM Transactions on Modeling and Computer Simulation*, vol. 8, no. 1, pp. 3–30, 1998, doi: 10.1145/272991.272995.

[7]Python Software Foundation, "random — Generate pseudo-random numbers," *Python 3.12 Documentation*. [En línea]. Disponible en: https://docs.python.org/3.12/library/random.html

8. Declaración de uso de Inteligencia Artificial

En la elaboración de esta práctica se utilizó la herramienta de inteligencia artificial Claude, modelo Claude Opus 5.5, desarrollado por Anthropic, como apoyo en la integración del código en Python, el reporte HTML con ilustraciones, los comentarios del código. El resto del desarrollo fue realizado por los integrantes. Todo el contenido fue revisado, verificado y ajustado por la autora, quien asume la responsabilidad sobre la versión final.

Herramienta: Claude Opus 5.5 (Anthropic).

Uso: integración y documentación del código.

Prompts utilizados:

1. Mejora visual: *«Implementa una vista dentro que permita visualizar de forma clara y explicativa como funciona esta práctica.»*

9. Anexos

- Repositorio grupal: https://github.com/ivecanahi/ape4-grupal.git
- Repositorio individual: https://github.com/Anthony-YP/Simulacion.git